Bloque 1. Arquitectura, configuración y administración de aplicaciones

Los servidores de aplicaciones permiten ejecutar aplicaciones web y proporcionar servicios que van más allá de la entrega de archivos estáticos. En este bloque se estudia su arquitectura, instalación, configuración y administración, utilizando Apache Tomcat como entorno de referencia.

1. Arquitectura y funcionamiento de los servidores de aplicaciones

1.1. ¿Qué es un servidor de aplicaciones?

Un servidor de aplicaciones es un software que proporciona un entorno de ejecución para aplicaciones, normalmente accesibles a través de una red. Se encarga de recibir peticiones, ejecutar la lógica de negocio y devolver los resultados correspondientes a los clientes.

A diferencia de un servidor que únicamente entrega archivos, un servidor de aplicaciones puede ejecutar código, gestionar sesiones, controlar el acceso de los usuarios y comunicarse con otros servicios, como bases de datos.

Entre sus principales responsabilidades se encuentran:

- Ejecutar la lógica de las aplicaciones web.
- Gestionar las peticiones HTTP recibidas.
- Administrar sesiones y mantener información de los usuarios.
- Proporcionar mecanismos de autenticación y autorización.
- Gestionar conexiones con bases de datos y otros servicios.
- Administrar los recursos necesarios para ejecutar las aplicaciones.
- Registrar errores y acontecimientos durante la ejecución.

1.2. Diferencias entre servidor web y servidor de aplicaciones

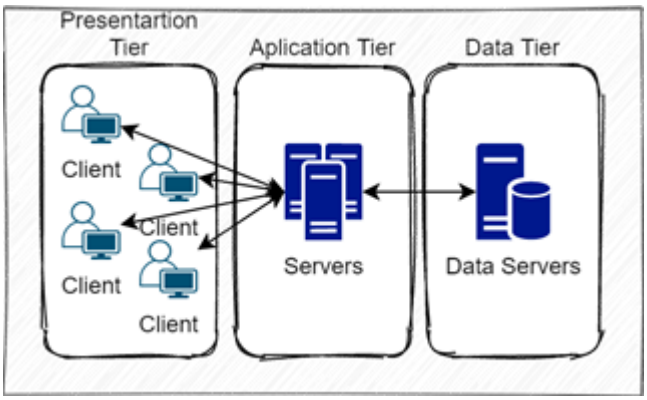
Aunque ambos pueden participar en la publicación de aplicaciones web, sus responsabilidades son diferentes.

Característica	Servidor web	Servidor de aplicaciones
Función principal	Recibir peticiones HTTP y entregar recursos.	Ejecutar aplicaciones y lógica de negocio.
Contenido habitual	HTML, CSS, JavaScript e imágenes.	Aplicaciones dinámicas y componentes ejecutables.
Procesamiento	Principalmente entrega de recursos estáticos y gestión de peticiones.	Ejecución de código y generación de respuestas dinámicas.
Sesiones	Puede gestionar determinadas funciones de sesión mediante módulos.	Proporciona mecanismos específicos para administrar sesiones de aplicación.
Ejemplos	Apache HTTP Server, Nginx.	Apache Tomcat, WildFly, Payara.

> **Importante:** un servidor web y un servidor de aplicaciones pueden trabajar conjuntamente. El primero puede encargarse de recibir las peticiones externas y el segundo de ejecutar la aplicación que genera las respuestas dinámicas.

1.3. Arquitectura de una aplicación web

En una arquitectura habitual, el navegador del usuario se comunica con un servidor web, que puede reenviar las peticiones dinámicas al servidor de aplicaciones. Este último ejecuta la aplicación y, cuando es necesario, consulta una base de datos.



Arquitectura de tres capas: presentación, aplicación y datos.

Podemos distinguir tres capas principales:

- **Capa de presentación:** representa la interfaz con la que interactúa el usuario, normalmente mediante un navegador web.
- **Capa de aplicación:** ejecuta la lógica de negocio, procesa las peticiones y aplica las reglas de funcionamiento de la aplicación.
- **Capa de datos:** almacena y proporciona acceso a la información mediante sistemas gestores de bases de datos.

1.4. Funcionamiento de una petición web

Cuando un usuario accede a una aplicación web, se produce una secuencia de comunicaciones entre los diferentes componentes.

1. El usuario introduce una dirección en el navegador.
2. El navegador establece una conexión con el servidor web.
3. El servidor web recibe la petición HTTP.
4. Si se trata de una petición dinámica, la reenvía al servidor de aplicaciones.
5. El servidor de aplicaciones ejecuta el componente correspondiente.
6. La aplicación puede consultar o modificar información en una base de datos.
7. El servidor genera una respuesta HTTP.
8. El navegador recibe y representa el resultado.

1.5. Apache Tomcat

Apache Tomcat es un servidor de aplicaciones de código abierto desarrollado por Apache Software Foundation. Está orientado a ejecutar aplicaciones web Java basadas en las tecnologías Servlet y Jakarta Server Pages (JSP).

Entre sus características destacan:

- Implementa las especificaciones Servlet y Jakarta Server Pages.
- Permite desplegar aplicaciones web empaquetadas en archivos WAR.
- Incluye un servidor HTTP integrado.
- Gestiona sesiones de usuario.
- Permite configurar usuarios, roles y restricciones de acceso.
- Proporciona herramientas de administración y monitorización.

> **Tomcat no es un servidor Jakarta EE completo.** Proporciona principalmente un contenedor de servlets y JSP. No incorpora todos los servicios empresariales que ofrecen plataformas como WildFly o Payara.

> ### Ejemplo: arquitectura con Apache y Tomcat
>
> Una empresa dispone de una aplicación web Java que utiliza una base de datos MariaDB. Para publicar la aplicación utiliza Apache HTTP Server y Tomcat.
>
> - Apache recibe las conexiones externas por el puerto 80 o 443.
> - Tomcat ejecuta la aplicación Java y escucha en el puerto 8080.
> - MariaDB almacena la información de la aplicación.
>
> El navegador accede a Apache, que reenvía las peticiones dinámicas a Tomcat. Tomcat procesa la solicitud y devuelve el resultado.

2. Instalación y configuración básica de Tomcat

2.1. Requisitos de instalación

Tomcat necesita un entorno de ejecución Java para funcionar. Por tanto, antes de instalarlo es necesario disponer de una versión del JDK o JRE compatible con la versión de Tomcat que se vaya a utilizar.

Los principales requisitos son:

- Un sistema operativo compatible, como Debian o Ubuntu.
- Una versión de Java compatible con Tomcat.
- Memoria y espacio de almacenamiento suficientes.
- Permisos adecuados para ejecutar y administrar el servicio.

2.2. Comprobar la instalación de Java

Antes de instalar Tomcat podemos comprobar si Java está disponible en el sistema.

```
java -version
```

El comando muestra la versión de Java instalada. Si el sistema no reconoce el comando, será necesario instalar un entorno Java.

Ejemplo: instalación de Java en Debian

```
sudo apt update
sudo apt install default-jdk

java -version
```

2.3. Instalación de Apache Tomcat en Debian

En Debian se puede instalar Tomcat utilizando los repositorios del sistema. El nombre del paquete depende de la versión disponible en la distribución.

```
sudo apt update
sudo apt install tomcat10
```

El paquete instala el servidor y configura su ejecución como servicio del sistema. La versión concreta debe comprobarse en los repositorios utilizados.

Para consultar el estado del servicio:

```
sudo systemctl status tomcat10
```

Para iniciar, detener o reiniciar el servicio:

```
sudo systemctl start tomcat10
sudo systemctl stop tomcat10
sudo systemctl restart tomcat10
```

Para configurar el inicio automático:

```
sudo systemctl enable tomcat10
```

2.4. Comprobar el funcionamiento

Por defecto, Tomcat suele utilizar el puerto TCP 8080 para recibir peticiones HTTP. Una vez iniciado el servicio, podemos acceder desde un navegador a la siguiente dirección:

```
http://localhost:8080
```

Si el servidor está correctamente instalado y configurado, se mostrará la página de bienvenida de Tomcat.

> **Comprobación desde otra máquina:** si Tomcat está instalado en una máquina virtual, se debe utilizar la dirección IP de esa máquina y comprobar que el puerto 8080 es accesible desde el equipo cliente.

2.5. Estructura de directorios

Tomcat organiza sus archivos en diferentes directorios, cada uno con una responsabilidad específica. La ubicación exacta depende del método de instalación y de la distribución.

Directorio	Función
`bin/`	Scripts y ejecutables para iniciar, detener y administrar Tomcat.
`conf/`	Archivos de configuración del servidor.
`lib/`	Bibliotecas compartidas utilizadas por Tomcat.
`logs/`	Registros de actividad, errores y acontecimientos.
`webapps/`	Directorio habitual de despliegue de aplicaciones web.
`work/`	Archivos temporales generados durante la ejecución.
`temp/`	Archivos temporales del servidor.

2.6. Archivos principales de configuración

La configuración de Tomcat se realiza principalmente mediante archivos XML situados en el directorio `conf/`.

Archivo	Función
`server.xml`	Configuración general del servidor, conectores, puertos y servicios.
`web.xml`	Configuración predeterminada de las aplicaciones web.
`tomcat-users.xml`	Definición de usuarios y roles de administración de Tomcat.
`context.xml`	Configuración de contextos y recursos de las aplicaciones.
`logging.properties`	Configuración del sistema de registros.

2.7. Configuración de puertos

Tomcat utiliza diferentes puertos para ofrecer sus servicios. Estos se configuran habitualmente mediante los conectores definidos en `server.xml`.

Puerto habitual	Servicio
8080	Conector HTTP para recibir peticiones web.
8443	Conector HTTPS cuando está configurado.
8005	Puerto de apagado local, si está habilitado.
8009	Puerto AJP, si se configura y habilita.

Los puertos pueden modificarse para evitar conflictos con otros servicios o adaptarse a la arquitectura de red.

Ejemplo: conector HTTP

El siguiente fragmento representa un conector HTTP configurado para escuchar en el puerto 8080:

```
<Connector
    port="8080"
    protocol="HTTP/1.1"
    connectionTimeout="20000"
    redirectPort="8443" />
```

2.8. Bibliotecas compartidas

Las bibliotecas contienen clases y componentes que pueden utilizarse durante la ejecución de las aplicaciones.

En Tomcat podemos distinguir:

- **Bibliotecas del servidor:** se encuentran en el directorio `lib/` y proporcionan funcionalidades comunes a Tomcat y a las aplicaciones.
- **Bibliotecas de una aplicación:** se almacenan normalmente en `WEB-INF/lib/` dentro de cada aplicación.
- **Clases propias de una aplicación:** se almacenan en `WEB-INF/classes/`.

> **Precaución:** no se deben instalar indiscriminadamente bibliotecas de aplicaciones en el directorio global de Tomcat. Una biblioteca compartida puede afectar a varias aplicaciones y provocar incompatibilidades entre versiones.

2.9. Registros del servidor

Los registros o logs permiten conocer qué sucede durante la ejecución del servidor. Son fundamentales para diagnosticar errores de configuración, problemas de conexión y fallos en las aplicaciones.

Entre los datos que pueden aparecer en los registros se encuentran:

- Inicio y parada del servidor.
- Errores al desplegar aplicaciones.
- Excepciones producidas durante la ejecución.
- Problemas de conexión y comunicación.

- Información sobre el funcionamiento de los componentes.

En Debian podemos consultar los registros mediante:

```
sudo journalctl -u tomcat10
sudo journalctl -u tomcat10 -f
```

También pueden consultarse los archivos del directorio de registros de Tomcat, cuya ubicación depende de la instalación.

3. Administración y despliegue de aplicaciones web

3.1. ¿Qué significa desplegar una aplicación?

El despliegue consiste en instalar y configurar una aplicación web en un servidor para que pueda ejecutarse y ser utilizada por los clientes.

El proceso suele incluir las siguientes operaciones:

1. Preparar y empaquetar la aplicación.
2. Copiar los archivos al servidor.
3. Configurar el contexto y los recursos necesarios.
4. Iniciar o actualizar la aplicación.
5. Comprobar que funciona correctamente.

3.2. Estructura de una aplicación web Java

Una aplicación web Java tradicional sigue una estructura de directorios definida por la especificación Servlet.

```
mi-aplicacion/
│
├── index.html
├── imagenes/
│   └── logo.png
│
└── WEB-INF/
    ├── web.xml
    │
    ├── classes/
    │   └── com/
    │       └── ejemplo/
    │           └── Controlador.class
    │
    └── lib/
        └── biblioteca.jar
```

Los elementos principales son:

- **Archivos públicos:** HTML, CSS, JavaScript e imágenes accesibles desde el navegador.
- **WEB-INF:** directorio protegido que contiene elementos internos de la aplicación.
- **web.xml:** descriptor de despliegue de la aplicación.
- **classes:** clases Java compiladas.
- **lib:** bibliotecas externas utilizadas por la aplicación.

> **Importante:** los archivos situados dentro de `WEB-INF` no pueden solicitarse directamente desde el navegador. El servidor los utiliza internamente para ejecutar la aplicación.

3.3. Archivos WAR

WAR (Web Application Archive) es el formato habitual para distribuir y desplegar aplicaciones web Java.

Un archivo WAR es un paquete comprimido que contiene la estructura de la aplicación, sus clases, bibliotecas y archivos de configuración.

Entre sus ventajas destacan:

- Facilita la distribución de aplicaciones.
- Permite agrupar todos los componentes en un único archivo.
- Facilita la instalación y actualización.
- Permite automatizar procesos de despliegue.

3.4. Directorio webapps

Tomcat utiliza habitualmente el directorio `webapps/` como ubicación para desplegar aplicaciones.

Cuando se copia un archivo WAR en este directorio, Tomcat puede detectar el nuevo paquete y desplegar automáticamente la aplicación, siempre que el despliegue automático esté habilitado.

Ejemplo: despliegue de una aplicación

Supongamos que disponemos del archivo `tienda.war`. Podemos copiarlo al directorio de aplicaciones de Tomcat.

```
sudo cp tienda.war /var/lib/tomcat10/webapps/
```

Una vez desplegada, normalmente podremos acceder a ella mediante:

```
http://localhost:8080/tienda
```

La ruta exacta del directorio de despliegue puede variar según la instalación y la distribución.

3.5. Actualización de aplicaciones

Cuando se publica una nueva versión de una aplicación, es necesario sustituir la versión anterior por la nueva.

El proceso de actualización puede incluir:

1. Realizar una copia de seguridad de la versión anterior.
2. Preparar el nuevo archivo WAR.
3. Sustituir el archivo anterior.
4. Permitir que Tomcat vuelva a desplegar la aplicación.
5. Comprobar su funcionamiento.

Precaución: antes de actualizar una aplicación en producción es necesario valorar la compatibilidad de los datos, las dependencias y los cambios en la configuración. Una actualización incorrecta puede provocar interrupciones del servicio.

3.6. Retirada de aplicaciones

La retirada de una aplicación consiste en eliminarla del servidor cuando deja de ser necesaria.

Para ello se debe detener o desinstalar correctamente la aplicación y eliminar sus archivos de despliegue, teniendo en cuenta los datos que deban conservarse.

Por ejemplo, para eliminar un archivo WAR:

```
sudo rm /var/lib/tomcat10/webapps/tienda.war
```

También se debe comprobar si existe un directorio desplegado asociado y si quedan recursos o configuraciones que deban retirarse.

3.7. Administración mediante herramientas gráficas

Tomcat dispone de aplicaciones de administración que permiten gestionar el despliegue y el estado de las aplicaciones desde un navegador.

Entre las herramientas disponibles se encuentran:

- **Manager App:** permite desplegar, detener, iniciar, recargar y retirar aplicaciones.
- **Host Manager:** permite administrar hosts virtuales de Tomcat.

Estas herramientas requieren configurar usuarios y roles adecuados. Además, su acceso suele estar restringido por motivos de seguridad.

4. Componentes web y configuración de aplicaciones

4.1. Servlets

Un Servlet es un componente Java que se ejecuta dentro de un contenedor de servlets, como el proporcionado por Tomcat. Su función principal es procesar peticiones y generar respuestas dinámicas.

Los servlets permiten recibir datos de formularios, consultar bases de datos, validar información y generar contenido para el navegador.

Su funcionamiento general comprende:

1. El navegador envía una petición HTTP.
2. Tomcat identifica el servlet asociado a la URL solicitada.
3. El contenedor ejecuta el método correspondiente.

4. El servlet procesa la petición.
5. Se genera una respuesta que Tomcat envía al navegador.

4.2. Ciclo de vida de un Servlet

El contenedor administra el ciclo de vida de los servlets. Los métodos principales son:

Método	Función
`init()`	Se ejecuta cuando se inicializa el servlet.
`service()`	Procesa las peticiones recibidas.
`doGet()`	Gestiona peticiones HTTP GET.
`doPost()`	Gestiona peticiones HTTP POST.
`destroy()`	Se ejecuta cuando el servlet va a ser retirado.

Ejemplo: Servlet básico

El siguiente ejemplo muestra un servlet que responde a una petición GET con un mensaje HTML.

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/saludo")
public class SaludoServlet extends HttpServlet {

    @Override
    protected void doGet(
        HttpServletRequest request,
        HttpServletResponse response
    ) throws ServletException, IOException {

        response.setContentType("text/html;charset=UTF-8");

        response.getWriter().println(
            "<h1>Hola desde Tomcat</h1>"
        );
    }
}
```

Si el servlet está correctamente desplegado, se podrá acceder a él mediante una URL similar a:

```
http://localhost:8080/mi-aplicacion/saludo
```

4.3. Jakarta Server Pages (JSP)

JSP es una tecnología que permite generar contenido dinámico mediante páginas que combinan HTML con elementos procesados en el servidor.

Cuando Tomcat recibe una petición a una página JSP, transforma su contenido en un servlet y lo ejecuta. El navegador recibe el resultado generado, no el código JSP original.

Ejemplo: página JSP

```
<%@ page contentType="text/html;charset=UTF-8" %>
<!DOCTYPE html>
<html>
<head>
    <title>Página JSP</title>
</head>
<body>

    <h1>Aplicación web</h1>

    <p>Fecha del servidor:</p>

    <%= new java.util.Date() %>

</body>
</html>
```

4.4. Configuración mediante web.xml

El archivo `web.xml` es el descriptor de despliegue de una aplicación web Java. Permite declarar componentes y establecer diferentes parámetros de configuración.

Entre sus posibilidades se encuentran:

- Definir servlets.
- Asociar servlets con determinadas URL.
- Establecer páginas de inicio.
- Configurar parámetros de inicialización.
- Definir tiempos de espera de las sesiones.
- Establecer restricciones de seguridad.
- Definir páginas de error.

Ejemplo: configuración de un Servlet en web.xml

```
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         version="6.0">

    <servlet>
        <servlet-name>SaludoServlet</servlet-name>
        <servlet-class>com.ejemplo.SaludoServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>SaludoServlet</servlet-name>
        <url-pattern>/saludo</url-pattern>
    </servlet-mapping>

</web-app>
```

En este ejemplo se declara un servlet y se establece que las peticiones a `/saludo` deben ser atendidas por él.

Nota sobre versiones: Tomcat 10 y versiones posteriores utilizan el espacio de nombres `jakarta.servlet`. Las aplicaciones antiguas basadas en Tomcat 9 utilizan `javax.servlet`. Ambas API no son directamente intercambiables.

4.5. Contextos de aplicación

Un contexto representa una aplicación web desplegada dentro de Tomcat. Cada aplicación dispone de una ruta de acceso propia, denominada ruta de contexto.

Por ejemplo, si desplegamos una aplicación llamada `tienda`, su URL habitual será:

```
http://localhost:8080/tienda
```

El contexto permite aislar la configuración y los recursos de cada aplicación. También puede utilizarse para establecer parámetros específicos, conexiones a bases de datos y otros recursos.

4.6. Recursos de las aplicaciones

Las aplicaciones web pueden necesitar recursos externos para funcionar, como conexiones a bases de datos, archivos, bibliotecas o servicios de otros sistemas.

Tomcat permite configurar recursos mediante diferentes mecanismos. Uno de ellos es JNDI, que proporciona un sistema de nombres para localizar recursos configurados en el servidor.

Algunos recursos habituales son:

- Conexiones a bases de datos mediante DataSource.
- Ficheros de configuración externos.
- Bibliotecas compartidas.
- Servicios proporcionados por otros sistemas.

Ejemplo: configuración de un DataSource

Una aplicación puede utilizar un recurso JNDI para obtener una conexión a una base de datos sin tener que gestionar directamente todos los parámetros de conexión.

```
<Resource
    name="jdbc/EmpresaDB"
    auth="Container"
    type="javax.sql.DataSource"
    driverClassName="org.mariadb.jdbc.Driver"
    url="jdbc:mariadb://localhost:3306/empresa"
    username="aplicacion"
    password="clave_segura"
    maxTotal="20"
    maxIdle="5" />
```

La aplicación puede localizar posteriormente el recurso mediante su nombre JNDI.

Seguridad de los recursos: las credenciales de acceso a bases de datos no deben almacenarse en repositorios públicos ni exponerse en archivos accesibles directamente desde el navegador. En producción deben utilizarse mecanismos adecuados de protección de secretos.

5. Ejercicios prácticos de aplicación

Ejercicio 1. Identificación de la arquitectura

Analiza una arquitectura formada por un navegador, Apache HTTP Server, Tomcat y MariaDB.

- Identifica la función de cada componente.
- Representa gráficamente el recorrido de una petición web.
- Explica qué componente procesa una petición dinámica.
- Indica qué sucedería si uno de los servicios deja de funcionar.

Ejercicio 2. Instalación y administración de Tomcat

Realiza la instalación de Tomcat en una máquina Debian y comprueba su funcionamiento.

- Comprueba la versión de Java.
- Instala e inicia Tomcat.
- Consulta el estado del servicio.
- Accede a la página de bienvenida.
- Localiza los principales directorios y archivos de configuración.
- Consulta los registros del servidor.

Ejercicio 3. Despliegue de aplicaciones

Prepara una aplicación web Java y realiza su despliegue en Tomcat.

- Identifica la estructura de directorios de la aplicación.
- Genera un archivo WAR.
- Despliega la aplicación en Tomcat.
- Comprueba el acceso desde un navegador.
- Realiza una actualización de la aplicación.
- Retira la aplicación del servidor.

Ejercicio 4. Componentes web

Desarrolla una pequeña aplicación que incluya una página JSP y un Servlet.

- Configura el acceso a un Servlet mediante una URL.
- Genera una respuesta dinámica.

- Utiliza un archivo `web.xml` para configurar el componente.
- Comprueba el funcionamiento de la aplicación.
- Identifica los archivos que forman parte del despliegue.

---

6. Resumen del bloque

- Un servidor de aplicaciones proporciona un entorno para ejecutar aplicaciones web y gestionar sus servicios.
- Apache HTTP Server y Apache Tomcat pueden trabajar conjuntamente para atender peticiones web.
- Tomcat es un contenedor de servlets y JSP que permite ejecutar aplicaciones Java.
- Los archivos de configuración, las bibliotecas y los registros son elementos fundamentales para administrar el servidor.
- Los archivos WAR permiten distribuir y desplegar aplicaciones web.
- Los servlets procesan peticiones HTTP y las páginas JSP generan contenido dinámico.
- Los contextos permiten organizar las aplicaciones desplegadas y configurar sus recursos.